Lctr 23: Calling ASM Functions in an ASM Function

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Why and how about calling ASM functions in an ASM function.
- Preparing arguments for function call:
  - Using LDR/LDRSH/LDRH/LDRSB/LDRB instructions for different type of arguments.
  - Using the LDRD instruction for 64-bit arguments.
- Getting the return value from the called function:
  - Using STR/STRH/STRB instructions for different type of returns.
  - Using the STRD instruction for a 64-bit return.

Demo project:

- Calling ASM functions to delegate 16-, 32-, and 64-bit operations.

C and ASM programming:

- Using the BL (branch with link) instruction to call ASM functions in ASM.
- Calling an ASM function defined in another ASM source file.

23.1 Introduction

So far, we have learned how to call an ASM function in C and how to preserve and restore the environment for the caller in the ASM callee so that the callee will not affect the working of the caller when it returns. Below are the points we have learned:

- We use R0 to R3 to pass the arguments to the callee, which uses R0 to return the result to the caller for situations where we use data types of up to 32 bits. For data types with more bits, we can combine the registers for arguments and the return value.
- If the callee uses registers in the function starting from R4, it needs to preserve these registers for the caller by using the PUSH / POP pair at the beginning and end of the function.
- When the function has done its work, it needs to hand the execution back to the caller by passing the value of the link register (LR) to the program counter (PC) using one of two approaches. The first is to use the `bx lr` instruction. The second is to pop the value of LR to PC if LR is preserved in the stack.

In this lecture, we will discuss how to call ASM functions in an ASM function. To this end, we will do the following:

- Prepare the arguments: put them in registers R0 to R3.
- Call the function by using the BL instruction, which will be introduced soon.
- If the callee has a return value, we need to save it to the corresponding variable.

When preparing arguments for function call, we will use the LDR, LDRSH, LDRH, LDRSB, and LDRB instructions to load different type of arguments when the arguments are no more than 32-bit wide. If the we use 64-bit arguments, we will use a new instruction, LDRD (load double word). In addition to load values, we often use pointers. All pointers are 32-bit wide.

When getting the return value from the called function, we use the STR, STRH, and STRB instructions for different type of returns that are no more than 32-bit wide. If the we use a 64-bit return value, we use the STRD (store double word) instruction, which will be discussed below.

23.2 LDRD and STRD instructions

In a previous lecture, we have discussed the following immediate-offset load instructions for 32-bit numbers:

Offset type	Instruction
Basic:	`LDR Rv, [Ra{, #os}]`
Pre-indexed:	`LDR Rv, [Ra, #os]!`
Post-indexed:	`LDR Rv, [Ra], #os`

It turns out that we have the LDRD instruction, used to load double words, 64-bit numbers, based on each of the above form:

Offset type	Instruction
Basic:	`LDRD Rv_L, Rv_H, [Ra{, #os}]`
Pre-indexed:	`LDRD Rv_L, Rv_H, [Ra, #os]!`
Post-indexed:	`LDRD Rv_L, Rv_H, [Ra], #os`

Here, `Rv_L` and `Rv_H` are the registers for the lower and higher words, respectively. Note that ARM use little endian by default, and `Rv_L` and `Rv_H` will load the words in the lower and higher addresses, respectively. Here we load 8 consecutive bytes in total; the first 4 to `Rv_L`, and the last 4 to `Rv_H`.

Note that there is no register-offset version of LDRD and STRD.

Example 1. Consider the memory map shown in Figure 1.

	Byte 0	Byte 1
0x2000_0200	0x70	0x72
0x2000_0202	0x74	0x76
0x2000_0204	0x78	0x7A
0x2000_0206	0x7C	0x7E
0x2000_0208	0x80	0x82
0x2000_020A	0x84	0x86
0x2000_020C	0x88	0x8A
0x2000_020E	0x8C	0x8E

Figure 1: Memory map.

Suppose the value of R0 is 0x2000_0200 in the following block of code. Determine the values of each involved registers after each instruction.

```
ldrd r2, r3, [r0, #8]
ldrd r4, r5, [r0, #4]!
ldrd r6, r7, [r0], #-4
```

Solution.

> -:-

We can revert the direction of the flow of data to get the STRD instructions, which are used to save two registers into the memory for 8-bytes using the lowest address of the data block.

Extending the idea of LDRD and STRD, we have LDM, load multiple, and STM, store multiple, instructions. These instructions are building blocks for the PUSH and POP instructions. We will not discuss the details of these instructions.

23.3 Calling ASM functions in ASM

There are similar terminologies for ASM functions when you read other books or documentations: functions, subroutines, routines, and procedures. They mean the same thing. We just use the term function here.

The reason for calling ASM functions in an ASM function is the same as for C functions to call C or ASM functions:

- Improved readability; this is especially true if we use meaningful function names.
- Remove duplicate code for improved maintainability.
- Improved quality using well-tested functions.

23.3.1 Caller vs callee

So far, all the ASM functions we have programmed do not call other functions. They are at the bottom of the call chain, and so they can be referred to as **leaf** functions.

A function that can call another function can be referred to as a caller or non-leaf function; here we use the term **stem** function.

A leaf function can only be a callee, and a stem function can be both a callee and a caller.

23.3.2 The BL instruction

Suppose we have two ASM functions: `asm_stem`, the caller function, and `asm_leaf`, the callee function.

To call `asm_leaf` in `asm_stem`, we use the BL (branch and link) instruction:

```
bl      asm_leaf
str     r0, [r4]
```

The label after the BL instruction is the name of the function to be called. This instruction does the following two operations:

- Set the link register (LR) to the address of the `str r0, [r4]` instruction, which immediately follows the BL instruction. Note that this STR instruction is here for illustration purpose only.
- Set the program counter (PC) to `asm_leaf`, the address of the very first instruction of the `asm_leaf` function.

Note that inside the `asm_leaf` function, we use the `bx lr` instruction at the end of the function to put the value of LR back to PC, so the execution will continue at the STR instruction after executing the `asm_leaf` function.

23.3.3 Preserving environment in the leaf function

According to EABI (embedded application binary interface), in `asm_leaf`, we can use R0 to R3 freely. If we use R4 to R12, we need to preserve their values for its caller, including `asm_stem`, by pushing them in the stack at the beginning of the function. At the end of the function, we need to recover their values by popping them out.

23.3.4 Preserving environment in the stem function

The preservation of the environment in the stem function `asm_stem` is more complicated.

First, `asm_stem` will be called by other functions, and hence we should preserve the environment for these callers, in the same way as what we do in the leaf function `asm_leaf`.

Second, we need to preserve the current link register (LR), as its value will be used to save another link when we call `asm_leaf`, as shown in Figure 2. Note that the `PUSH {LR}` instruction is required in `asm_stem` and optional in `asr_leaf`.

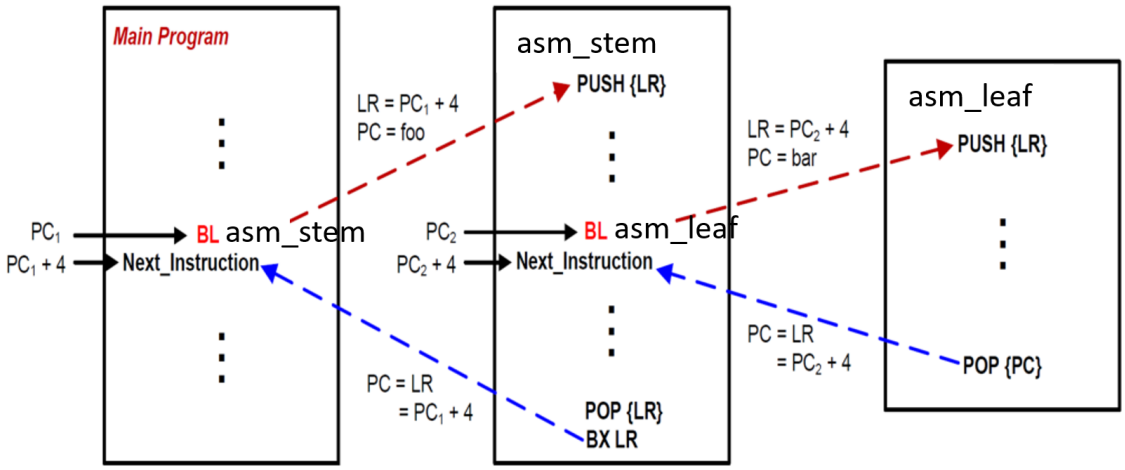

Figure 2: Memory map.

Third, we need to preserve the arguments passed to this function from its caller. These arguments are saved in R0 to R3. Yet, when we call `asm_leaf`, we need to use these registers to hold the arguments for `asm_leaf`. This is related to the topic we address in the next subsection.

23.3.5 Preparing arguments for the callee

To call `asm_leaf` in `asm_stem`, we need to prepare all the arguments used by `asm_leaf`. This is done by using the following instructions:

- The LDR/LDRSH/LDRH/LDRSB/LDRB instructions for different types of arguments up to 32-bit wide.
- The LDRD instruction for 64-bit arguments.

Some times we need to use pointers; they are all 32-bit wide for 32-bit ARM processors.

23.3.6 Getting the return value from the callee

After calling `asm_leaf`, the first thing `asm_stem` does is to save the return value before it gets lost. We need to use one of the following instructions:

- The STR/STRH/STRB instructions for a return from R0 according to the data type.
- The STRD instruction for a 64-bit return from R0 and R1.

23.4 Demo project

23.4.1 Leaf function for the modulo operation

The first leaf function, which performs the modulo operation `x % y` on the two parameters `x` and `y`, is shown below.

```
@ 32-bit unsigned modulo operation: x % y
@ uint32_t mp_umod_32bit_s(uint32_t x, uint32_t y)
@   Input:
@     * x -> r0
@     * y -> r1
@   Output:
@     * r0
mp_umod_32bit_s:
@ Implemented as: x % y = x - (x/y)*y
    udiv    r2, r0, r1
    mls     r0, r1, r2, r0
    bx      lr
```

Note that we only use 2 registers for arguments and one for temporary variable. Hence, there is no need to preserve the environment for the caller.

23.4.2 Stem function for the 32-bit modulo operation on an array

First, consider the following problem. Let `x`, `y`, and `z` be three `uint32_t` arrays, each having `n` elements. We want to calculate the following:

```
z[i] = x[i] % y[i], for i = 0, 1, ..., n-1
```

We can program this function as a leaf function. Yet, since we have the tested code of `mp_umod_32bit_s`, we want to take advantage of it. So, we only need to program a stem function to handle the loop, within which we need to prepare the arguments for `mp_umod_32bit_s` and get the returned value from it.

This function has the following prototype:

```
void mp_umod_32bit_array_s(uint32_t *x, uint32_t *y, uint32_t
    *z, int n);
```

It is saved in `gm1s_calling_asm_fn_in_asm_fn.s` in the `gm1s_calling_asm_fn_in_asm_fn` project. The code of the function is shown below:

```
@ 32-bit unsigned array modulo (remainder) operation
@ void mp_umod_32bit_array_s(uint32_t *x, uint32_t *y, uint32_t *z, int n);
@   Input:
@     * x -> r0
@     * y -> r1
@     * z -> r2
@     * n -> r3
@   Output:
@     * none
@   Others:
@     * copy values of x, y, z, n to r4 to r7.
@         * will use r0 to r3 for arguments for fn call
@     * int i -> r8 for loop control.

mp_umod_32bit_array_s:
    @ Reserve the needed registers
    push    {r4-r8, lr}
    mov     r4, r0
    mov     r5, r1
    mov     r6, r2
    mov     r7, r3

    ldr     r8, =0
mp_umod_32bit_array_s_loop:
    cmp     r8, r7      @ (i < n)?
    bge     mp_umod_32bit_array_s_end_loop
        @ Prepare to call mp_umod_32bit_s(x, y);
        ldr     r0, [r4, r8, lsl #2]    @ Argument: x -> r0
        ldr     r1, [r5, r8, lsl #2]    @ Argument: y -> r1
        bl      mp_umod_32bit_s
        str     r0, [r6, r8, lsl #2]    @ Save return value
        add     r8, #1
    b       mp_umod_32bit_array_s_loop

mp_umod_32bit_array_s_end_loop:
    pop     {r4-r8, pc}
```

23.4.3 Stem function for the 16-bit modulo operation on an array

Now, consider the case that `x`, `y`, and `z` are `uint16_t` arrays, each having `n` elements. We want to perform the same functionality as

```
void mp_umod_32bit_array_s(uint32_t *x, uint32_t *y, uint32_t
    *z, int n);
```

Now, the prototype will be changed to

```
void mp_umod_16bit_array_s(uint16_t *x, uint16_t *y, uint16_t
    *z, int n);
```

The real difference between this function and the previous one is illustrated below:

```
        @ Prepare to call mp_umod_32bit_s(x, y);
        ldrh    r0, [r4, r8, lsl #1]    @ Argument: x -> r0
        ldrh    r1, [r5, r8, lsl #1]    @ Argument: y -> r1
        bl      mp_umod_32bit_s
        strh    r0, [r6, r8, lsl #1]    @ Save return value
```

We can see that we have only changed the LDR/STR instructions as the data type has been changed. We still call the same leaf function since it can handle numbers up to 32 bits.

23.4.4 Leaf function for the 64-bit addition

The second leaf function, which performs the 64-bit addition operation `x + y` on the two parameters `x` and `y`, is shown below.

```
@ 64-bit unsigned/signed addition: x + y
@ int64_t mp_add_64bit_s(int64_t x, int64_t y)
@   Input:
@     * x -> r1:r0
@     * y -> r3:r2
@   Output:
@     * r1:r0
mp_add_64bit_s:
    adds    r0, r2
    adc     r1, r3
    bx      lr
```

23.4.5 Stem function for the 64-bit addition operation on an array

Now, let's consider the following 64-bit addition problem. Let `x`, `y`, and `z` be three `int64_t` arrays, each having `n` elements. We want to calculate the following:

```
z[i] = x[i] + y[i], for i = 0, 1, ..., n-1
```

Again, we want to perform the above element-based operation using existing a well-tested leaf function; it is `mp_add_64bit_s(int64_t x, int64_t y)`.

We can use the following prototype for this stem function:

```
void mp_add_64bit_array_s(int64_t *x, int64_t *y, int64_t *z,
    int n);
```

The code is given below:

```
@ void mp_add_64bit_array_s(int64_t *x, int64_t *y, int64_t *z
    , int n);
@   Input:
@     * x -> r0
@     * y -> r1
@     * z -> r2
@     * n -> r3
@   Output:
@     * none
@   Others:
@     * save values of x, y, z, n to r4 to r7.
@       * use r0 to r3 for parameters for function call.

mp_add_64bit_array_s:
    @ Reserve the needed registers
    push    {r4-r7, lr}
    mov     r4, r0
    mov     r5, r1
    mov     r6, r2
    mov     r7, r3

    @ We can use r7 as the loop control register
mp_add_64bit_array_s_loop:
    cmp     r7, #0      @ (n > 0)?
    ble     mp_add_64bit_array_s_end_loop
```

```
        @ Prepare to call mp_add_64bit_array_s(int64_t x, int64_t y);
        ldrd    r0, r1, [r4], #8        @ Prepare x -> r1:r0
        ldrd    r2, r3, [r5], #8        @ Prepare y -> r3:r2
        bl      mp_add_64bit_s
        strd    r0, r1, [r6], #8        @ Save return value
        sub     r7, #1
    b       mp_add_64bit_array_s_loop

mp_add_64bit_array_s_end_loop:
    pop     {r4-r7, pc}
```